

RESTAURANT MANAGEMENT SYSTEM

Database Management Project - Phase 3

SQL Features, Queries, Executed Outputs, and Final Summary

Item	Description
Course	Database Management Systems
Project Phase	Phase 3 - Feature Queries and Output Documentation
Project Type	Restaurant Management System
Prepared By	Aliya Nadeem, Hira Fatima & Hina Tariq (Group F)
Submitted To	Mam Ambreen Akmal
Submission Date	Wednesday -10 th -June

In This document We present a complete Phase 3 database project report for a restaurant management system. It includes the database design, core modules, at least fifteen practical features, SQL queries for each feature, and screenshots of the executed outputs.

1. Project Abstract

The Restaurant Management System is a relational database project designed to manage restaurant operations such as customer records, reservations, dining tables, staff, orders, order items, menu items, payments, suppliers, inventory, purchase orders, and customer feedback. The purpose of Phase 3 is to demonstrate how the database supports real business features through executable SQL queries and useful outputs.

The system stores data in normalized tables and connects them through primary keys and foreign keys. This makes the data reliable, reduces duplication, and allows the restaurant to generate reports quickly. The project is suitable for a dine-in, takeaway, and delivery restaurant where management needs accurate information for billing, service, stock control, staff performance, and decision-making.

1.1 Phase 3 Objectives

- List the major functional features of the restaurant management database.
- Write SQL queries for each selected feature and show the executed output.
- Use joins, grouping, aggregation, filtering, common table expressions, and window functions where suitable.
- Show how the system supports daily operations such as reservations, billing, inventory alerts, and payments.
- Provide a final summary explaining the value and future improvement scope of the system.

1.2 Important Note About Execution

The SQL output screenshots in this report were generated from an actually executed sample restaurant database. The demonstration database uses SQLite for local execution, while the design follows standard relational database principles and can be implemented in PostgreSQL, MySQL, SQL Server, or Oracle with minor syntax adjustments for date and formatting functions.

2. System Scope and Main Modules

The system is designed for restaurant owners, managers, cashiers, waiters, kitchen staff, and hosts. It supports the full flow of a restaurant transaction: a customer can reserve a table, visit the restaurant, place an order, receive food, pay the bill, and submit feedback. At the same time, management can monitor stock, suppliers, staff performance, and revenue trends.

2.1 Main Users

User Role	Main Responsibilities in the System
Manager	Reviews sales, staff performance, inventory, supplier purchases, and feedback reports.
Cashier	Generates bills, records payments, checks pending payments, and reconciles daily collections.
Waiter	Creates orders, adds order items, tracks order status, and serves tables.
Host	Manages table reservation, table allocation, and customer seating.
Kitchen Staff	Uses order details and menu items to prepare food and monitor ingredient needs.

2.2 Functional Modules

- Customer management and loyalty history.
- Reservation management and table allocation.
- Menu category and menu item management.
- Order entry, order item detail, and bill calculation.
- Payment processing and payment-method reporting.
- Inventory stock monitoring and low-stock alerts.
- Supplier and purchase order management.
- Staff performance and service reporting.
- Feedback management and customer satisfaction analysis.

3. Phase 3 Feature List

The following fifteen features were selected for Phase 3. Each feature is demonstrated later with an SQL query and an executed output screenshot.

ID	Feature	Purpose
F01	Active Menu Catalogue	Shows all active menu items grouped by category so the restaurant can maintain the customer-facing menu and quickly verify prices.
F02	Reservation and Table Allocation	Lists reservations for a selected date with table number and capacity, helping the host avoid double booking and seat customers correctly.
F03	Order Bill Calculation	Calculates each line item and the complete bill total for a specific order, which is needed by waiters and cashiers.
F04	Daily Sales Report	Summarizes total orders, item quantity, and gross sales by date for management reporting and closing-day reconciliation.
F05	Best-Selling Menu Items	Finds the most popular dishes by units sold and revenue, supporting menu planning, promotions, and ingredient purchase decisions.
F06	Category Revenue Analysis	Shows how much revenue each food category contributes, helping managers decide which menu groups deserve focus.
F07	Customer Loyalty Ranking	Ranks customers by lifetime spending and visits, helping the restaurant identify loyal customers for discounts and offers.
F08	Inventory Low-Stock Alerts	Detects ingredients below or equal to reorder level so the kitchen can avoid stockouts during service hours.
F09	Supplier Restocking Summary	Summarizes purchase orders by supplier to track supplier dependency, ordering frequency, and total purchase value.
F10	Staff Performance Report	Measures orders and revenue handled by each staff member, supporting performance review and shift planning.
F11	Payment Method Analysis	Breaks paid transactions into cash, card, and digital wallet amounts to support cashier reconciliation.
F12	Table Utilization Report	Shows which tables generate the most orders and revenue so managers can understand seating performance.
F13	Pending and Unpaid Order Monitoring	Identifies orders that are still open, preparing, or not fully paid so the cashier can follow up before closing.
F14	Monthly Revenue Trend	Compares monthly sales and average bill size, giving management a high-level view of business growth.
F15	Customer Feedback Insights	Groups feedback by category to identify customer satisfaction patterns and improvement areas.

4. Database Design and Table Dictionary

The database is normalized around the restaurant transaction cycle. Each table stores one main entity, and relationships are created through foreign keys. This structure allows the system to answer operational and managerial questions through SQL joins and reports.

4.1 Core Operational Tables

Table	Description
restaurant	Stores restaurant branch profile such as name, address, and phone.
customer	Stores customer identity, contact details, email, and joining date.
staff	Stores employee details including role, shift, and salary.
dining_table	Stores restaurant table number, seating capacity, location, and current status.
menu_category	Stores categories such as Starters, BBQ, Rice, Desserts, Beverages, and Deals.
menu_item	Stores food item name, category, price, preparation time, and active status.
reservation	Stores customer reservations with date, time, table, party size, and status.

4.2 Transaction, Inventory, and Reporting Tables

Table	Description
orders	Stores customer orders, order type, date/time, staff member, table, and order status.
order_item	Stores each item inside an order with quantity and selling price.
payment	Stores payment method, amount, date, and payment status for each order.
supplier	Stores supplier name, city, and phone number.
inventory_item	Stores ingredient stock, unit, reorder level, cost, and linked supplier.
purchase_order	Stores restocking orders placed to suppliers.
purchase_order_item	Stores ingredients and quantities included in a purchase order.
feedback	Stores rating and comments submitted by customers after orders.

5. Keys, Relationships, and Business Rules

5.1 Primary Keys and Foreign Keys

Key	Relationship Use
customer.customer_id	Primary key used by reservation, orders, and feedback.
dining_table.table_id	Primary key used by reservation and orders.
staff.staff_id	Primary key used by orders to show who handled the order.
menu_category.category_id	Primary key used by menu_item.
menu_item.item_id	Primary key used by order_item.
orders.order_id	Primary key used by order_item, payment, and feedback.
supplier.supplier_id	Primary key used by inventory_item and purchase_order.
purchase_order.po_id	Primary key used by purchase_order_item.

5.2 Business Rules

- A customer can have many reservations and many orders.
- One table can be reserved many times on different dates and times.
- One order can contain many order items, but each order item belongs to one order.
- One menu category can contain many menu items.
- Payment is linked to the order for billing and reconciliation.
- Inventory items are linked to suppliers so low stock can trigger restocking decisions.
- Feedback is linked to a completed order and customer to support satisfaction analysis.

6. Entity Relationship Diagram

The ERD below shows the main relational structure of the Restaurant Management System. It highlights one-to-many relationships between customers and orders, orders and order items, menu categories and menu items, suppliers and inventory, and purchase orders and purchase order items.

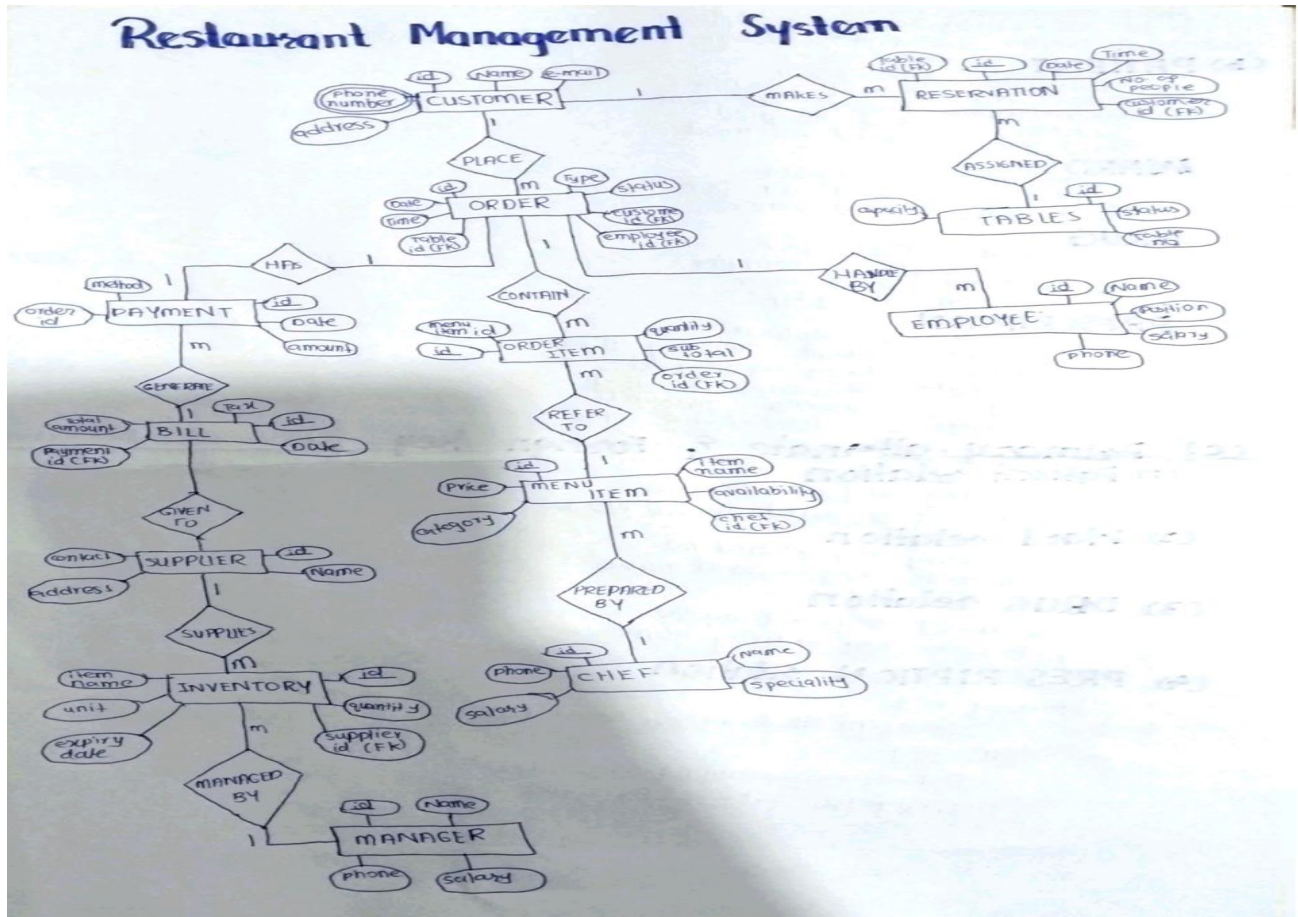


Figure 1: Entity Relationship Diagram of the Restaurant Management System

7. SQL Implementation Overview

The project uses a relational schema with primary keys, foreign keys, not-null constraints, and check constraints where needed. The full sample database contains realistic data for customers, reservations, tables, staff, menu items, orders, payments, suppliers, inventory, purchase orders, and feedback.

7.1 Main SQL DDL Excerpt

DDL Component	SQL Statement
Orders and Order Items	<pre> CREATE TABLE orders (order_id INTEGER PRIMARY KEY, customer_id INTEGER, table_id INTEGER, staff_id INTEGER NOT NULL, order_date TEXT NOT NULL, order_time TEXT NOT NULL, order_type TEXT NOT NULL, status TEXT NOT NULL, FOREIGN KEY (customer_id) REFERENCES customer(customer_id), FOREIGN KEY (table_id) REFERENCES dining_table(table_id), FOREIGN KEY (staff_id) REFERENCES staff(staff_id)); CREATE TABLE order_item (order_item_id INTEGER PRIMARY KEY, order_id INTEGER NOT NULL, item_id INTEGER NOT NULL, quantity INTEGER NOT NULL, unit_price REAL NOT NULL, FOREIGN KEY (order_id) REFERENCES orders(order_id), FOREIGN KEY (item_id) REFERENCES menu_item(item_id)); </pre>

7.2 Sample Data Used for Query Execution

Entity	Sample Record Count
Customers	12
Staff	8
Dining Tables	12
Menu Items	25
Reservations	8
Orders	28
Order Items	80
Payments	27
Suppliers	5
Inventory Items	14
Purchase Orders	6
Feedback Records	18

8.1 Feature F01: Active Menu Catalogue

Shows all active menu items grouped by category so the restaurant can maintain the customer-facing menu and quickly verify prices.

SQL Query

Field	Details
Feature ID	F01
Feature Name	Active Menu Catalogue
Purpose	Shows all active menu items grouped by category so the restaurant can maintain the customer-facing menu and quickly verify prices.
SQL Query	<pre>SELECT mc.category_name, mi.item_name, printf("%.2f", mi.price) AS price, mi.prep_time_min AS prep_min, CASE WHEN mi.is_active = 1 THEN 'Available' ELSE 'Hidden' END AS status FROM menu_item mi JOIN menu_category mc ON mc.category_id = mi.category_id WHERE mi.is_active = 1 ORDER BY mc.category_name, mi.item_name LIMIT 15;</pre>

Executed Output Screenshot

SQL Output Screenshot - F01

Active Menu Catalogue

category_name	item_name	price	prep_min	status
BBQ	BBQ Platter	3200.00	35	Available
BBQ	Beef Seekh Kebab	650.00	16	Available
BBQ	Chicken Tikka	520.00	18	Available
BBQ	Malai Boti	780.00	19	Available
Beverages	Fresh Lime	260.00	4	Available
Beverages	Mineral Water	120.00	1	Available
Beverages	Mint Margarita	350.00	5	Available
Beverages	Soft Drink	180.00	2	Available
Deals	Family Deal 1	3999.00	35	Available
Deals	Lunch Combo	1299.00	22	Available
Deals	Student Deal	999.00	18	Available
Desserts	Cheesecake Slice	720.00	8	Available
Desserts	Chocolate Lava Cake	620.00	10	Available
Desserts	Kheer Cup	300.00	6	Available
Main Course	Beef Steak	2250.00	28	Available

Rows shown: 15 | Output generated from executed SQL on sample restaurant database.

Figure 2: Executed SQL output for F01 - Active Menu Catalogue (15 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.2 Feature F02: Reservation and Table Allocation

Lists reservations for a selected date with table number and capacity, helping the host avoid double booking and seat customers correctly.

SQL Query

Field	Details
Feature ID	F02
Feature Name	Reservation and Table Allocation
Purpose	Lists reservations for a selected date with table number and capacity, helping the host avoid double booking and seat customers correctly.
SQL Query	<pre>SELECT r.reservation_id, c.customer_name, dt.table_no, dt.capacity, r.reservation_time, r.party_size, r.status FROM reservation r JOIN customer c ON c.customer_id = r.customer_id JOIN dining_table dt ON dt.table_id = r.table_id WHERE r.reservation_date = '2026-05-10' ORDER BY r.reservation_time;</pre>

Executed Output Screenshot

SQL Output Screenshot - F02

Reservation and Table Allocation

reservation_id	customer_name	table_no	capacity	reservation_time	party_size	status
3	Usman Tariq	T02	2	18:30	2	Seated
1	Ayesha Khan	T05	4	19:00	4	Confirmed
2	Sara Ahmed	T08	8	20:00	7	Confirmed

Rows shown: 3 | Output generated from executed SQL on sample restaurant database.

Figure 3: Executed SQL output for F02 - Reservation and Table Allocation (3 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.3 Feature F03: Order Bill Calculation

Calculates each line item and the complete bill total for a specific order, which is needed by waiters and cashiers.

SQL Query

Field	Details
Feature ID	F03
Feature Name	Order Bill Calculation
Purpose	Calculates each line item and the complete bill total for a specific order, which is needed by waiters and cashiers.
SQL Query	<pre>SELECT o.order_id, c.customer_name, dt.table_no, mi.item_name, oi.quantity, printf("%.2f", oi.unit_price) AS unit_price, printf("%.2f", oi.quantity * oi.unit_price) AS line_total, printf("%.2f", SUM(oi.quantity * oi.unit_price) OVER (PARTITION BY o.order_id)) AS order_total FROM orders o JOIN customer c ON c.customer_id = o.customer_id JOIN dining_table dt ON dt.table_id = o.table_id JOIN order_item oi ON oi.order_id = o.order_id JOIN menu_item mi ON mi.item_id = oi.item_id WHERE o.order_id = 1007 ORDER BY mi.item_name;</pre>

Executed Output Screenshot

SQL Output Screenshot - F03

Order Bill Calculation

order_id	customer_name	table_no	item_name	quantity	unit_price	line_total	order_total
1007	Ayesha Khan	T06	Beef Seekh Kebab	2	650.00	1300.00	5020.00
1007	Ayesha Khan	T06	Cheesecake Slice	1	720.00	720.00	5020.00
1007	Ayesha Khan	T06	Chicken Karahi Half	1	1450.00	1450.00	5020.00
1007	Ayesha Khan	T06	Mint Margarita	3	350.00	1050.00	5020.00
1007	Ayesha Khan	T06	Plain Rice	2	250.00	500.00	5020.00

Rows shown: 5 | Output generated from executed SQL on sample restaurant database.

Figure 4: Executed SQL output for F03 - Order Bill Calculation (5 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.4 Feature F04: Daily Sales Report

Summarizes total orders, item quantity, and gross sales by date for management reporting and closing-day reconciliation.

SQL Query

Field	Details
Feature ID	F04
Feature Name	Daily Sales Report
Purpose	Summarizes total orders, item quantity, and gross sales by date for management reporting and closing-day reconciliation.
SQL Query	<pre>SELECT o.order_date, COUNT(DISTINCT o.order_id) AS total_orders, SUM(oi.quantity) AS items_sold, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS gross_sales FROM orders o JOIN order_item oi ON oi.order_id = o.order_id WHERE o.status <> 'Cancelled' GROUP BY o.order_date ORDER BY o.order_date;</pre>

Executed Output Screenshot

SQL Output Screenshot - F04

Daily Sales Report

order_date	total_orders	items_sold	gross_sales
2026-04-27	2	10	5890.00
2026-04-28	1	7	4520.00
2026-04-29	1	5	4479.00
2026-05-01	1	5	3610.00
2026-05-02	1	6	3537.00
2026-05-03	2	17	8140.00
2026-05-04	1	4	2838.00
2026-05-05	1	7	7330.00
2026-05-06	1	9	6279.00
2026-05-07	2	10	5530.00
2026-05-08	2	10	6539.00
2026-05-09	1	5	5200.00
2026-05-10	2	15	13928.00
2026-05-11	1	4	2439.00
2026-05-12	1	7	5750.00

Rows shown: 15 | Output generated from executed SQL on sample restaurant database.

Figure 5: Executed SQL output for F04 - Daily Sales Report (22 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.5 Feature F05: Best-Selling Menu Items

Finds the most popular dishes by units sold and revenue, supporting menu planning, promotions, and ingredient purchase decisions.

SQL Query

Field	Details
Feature ID	F05
Feature Name	Best-Selling Menu Items
Purpose	Finds the most popular dishes by units sold and revenue, supporting menu planning, promotions, and ingredient purchase decisions.
SQL Query	<pre>SELECT mi.item_name, SUM(oi.quantity) AS units_sold, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS revenue FROM order_item oi JOIN menu_item mi ON mi.item_id = oi.item_id JOIN orders o ON o.order_id = oi.order_id WHERE o.status <> 'Cancelled' GROUP BY mi.item_name ORDER BY units_sold DESC, revenue DESC LIMIT 10;</pre>

Executed Output Screenshot

SQL Output Screenshot - F05

Best-Selling Menu Items

item_name	units_sold	revenue
Mint Margarita	22	7700.00
Soft Drink	19	3420.00
Fresh Lime	15	3900.00
Mineral Water	14	1680.00
Chicken Biryani	9	5850.00
Kheer Cup	9	2700.00
Plain Rice	9	2250.00
Beef Seekh Kebab	7	4550.00
Student Deal	6	5994.00
Chocolate Lava Cake	5	3100.00

Rows shown: 10 | Output generated from executed SQL on sample restaurant database.

Figure 6: Executed SQL output for F05 - Best-Selling Menu Items (10 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.6 Feature F06: Category Revenue Analysis

Shows how much revenue each food category contributes, helping managers decide which menu groups deserve focus.

SQL Query

Field	Details
Feature ID	F06
Feature Name	Category Revenue Analysis
Purpose	Shows how much revenue each food category contributes, helping managers decide which menu groups deserve focus.
SQL Query	<pre>SELECT mc.category_name, COUNT(DISTINCT o.order_id) AS orders_count, SUM(oi.quantity) AS quantity_sold, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS revenue FROM menu_category mc JOIN menu_item mi ON mi.category_id = mc.category_id JOIN order_item oi ON oi.item_id = mi.item_id JOIN orders o ON o.order_id = oi.order_id WHERE o.status <> 'Cancelled' GROUP BY mc.category_name ORDER BY SUM(oi.quantity * oi.unit_price) DESC;</pre>

Executed Output Screenshot

SQL Output Screenshot - F06

Category Revenue Analysis

category_name	orders_count	quantity_sold	revenue
Deals	9	14	29886.00
Main Course	11	15	25070.00
BBQ	9	18	19350.00
Beverages	24	70	16700.00
Rice	11	21	11250.00
Desserts	11	18	8680.00
Starters	4	6	4320.00

Rows shown: 7 | Output generated from executed SQL on sample restaurant database.

Figure 7: Executed SQL output for F06 - Category Revenue Analysis (7 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.7 Feature F07: Customer Loyalty Ranking

Ranks customers by lifetime spending and visits, helping the restaurant identify loyal customers for discounts and offers.

SQL Query

Field	Details
Feature ID	F07
Feature Name	Customer Loyalty Ranking
Purpose	Ranks customers by lifetime spending and visits, helping the restaurant identify loyal customers for discounts and offers.
SQL Query	<pre>SELECT c.customer_id, c.customer_name, COUNT(DISTINCT o.order_id) AS visits, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS total_spent, MAX(o.order_date) AS last_visit FROM customer c JOIN orders o ON o.customer_id = c.customer_id JOIN order_item oi ON oi.order_id = o.order_id WHERE o.status <> 'Cancelled' GROUP BY c.customer_id, c.customer_name ORDER BY SUM(oi.quantity * oi.unit_price) DESC LIMIT 10;</pre>

Executed Output Screenshot

SQL Output Screenshot - F07

Customer Loyalty Ranking

customer_id	customer_name	visits	total_spent	last_visit
7	Maha Raza	2	13978.00	2026-05-10
3	Sara Ahmed	3	13239.00	2026-06-02
1	Ayesha Khan	3	13160.00	2026-06-01
9	Hira Shah	2	13080.00	2026-05-12
11	Noor Bukhari	3	12517.00	2026-06-04
6	Usman Tariq	3	11377.00	2026-06-03
5	Fatima Noor	2	8810.00	2026-05-09
4	Bilal Malik	2	7018.00	2026-05-08
2	Hamza Ali	2	6070.00	2026-05-07
10	Zain Qureshi	1	6000.00	2026-05-13

Rows shown: 10 | Output generated from executed SQL on sample restaurant database.

Figure 8: Executed SQL output for F07 - Customer Loyalty Ranking (10 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.8 Feature F08: Inventory Low-Stock Alerts

Detects ingredients below or equal to reorder level so the kitchen can avoid stockouts during service hours.

SQL Query

Field	Details
Feature ID	F08
Feature Name	Inventory Low-Stock Alerts
Purpose	Detects ingredients below or equal to reorder level so the kitchen can avoid stockouts during service hours.
SQL Query	<pre>SELECT ii.item_name, ii.stock_qty, ii.reorder_level, ii.unit, s.supplier_name, ROUND(ii.reorder_level - ii.stock_qty, 2) AS shortage_qty FROM inventory_item ii LEFT JOIN supplier s ON s.supplier_id = ii.supplier_id WHERE ii.stock_qty <= ii.reorder_level ORDER BY shortage_qty DESC, ii.item_name;</pre>

Executed Output Screenshot

SQL Output Screenshot - F08

Inventory Low-Stock Alerts

item_name	stock_qty	reorder_level	unit	supplier_name	shortage_qty
Chicken	22.0	30.0	kg	Meat Master Suppliers	8.0
Fresh Mint	8.0	15.0	bunch	FreshFarm Produce	7.0
Tomatoes	18.0	25.0	kg	FreshFarm Produce	7.0
Beef	14.0	20.0	kg	Meat Master Suppliers	6.0
Mineral Water	9.0	15.0	crate	Beverage Hub	6.0
Cheese	7.0	10.0	kg	Daily Dairy Co.	3.0
Chocolate	6.0	8.0	kg	Bakery Base Foods	2.0
Mutton	10.0	12.0	kg	Meat Master Suppliers	2.0
Spices Mix	5.0	7.0	kg	FreshFarm Produce	2.0

Rows shown: 9 | Output generated from executed SQL on sample restaurant database.

Figure 9: Executed SQL output for F08 - Inventory Low-Stock Alerts (9 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.9 Feature F09: Supplier Restocking Summary

Summarizes purchase orders by supplier to track supplier dependency, ordering frequency, and total purchase value.

SQL Query

Field	Details
Feature ID	F09
Feature Name	Supplier Restocking Summary
Purpose	Summarizes purchase orders by supplier to track supplier dependency, ordering frequency, and total purchase value.
SQL Query	<pre>SELECT s.supplier_name, COUNT(DISTINCT po.po_id) AS purchase_orders, SUM(poi.quantity) AS total_units_ordered, printf("%.2f", SUM(poi.quantity * poi.unit_cost)) AS purchase_value, MAX(po.order_date) AS last_order_date FROM supplier s JOIN purchase_order po ON po.supplier_id = s.supplier_id JOIN purchase_order_item poi ON poi.po_id = po.po_id GROUP BY s.supplier_name ORDER BY SUM(poi.quantity * poi.unit_cost) DESC;</pre>

Executed Output Screenshot

SQL Output Screenshot - F09

Supplier Restocking Summary

supplier_name	purchase_orders	total_units_ordered	purchase_value	last_order_date
Meat Master Suppliers	2	105.0	97350.00	2026-05-09
Beverage Hub	1	45.0	42500.00	2026-05-01
Bakery Base Foods	1	70.0	27200.00	2026-05-07
Daily Dairy Co.	1	52.0	25300.00	2026-05-04
FreshFarm Produce	1	100.0	17150.00	2026-04-25

Rows shown: 5 | Output generated from executed SQL on sample restaurant database.

Figure 10: Executed SQL output for F09 - Supplier Restocking Summary (5 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.10 Feature F10: Staff Performance Report

Measures orders and revenue handled by each staff member, supporting performance review and shift planning.

SQL Query

Field	Details
Feature ID	F10
Feature Name	Staff Performance Report
Purpose	Measures orders and revenue handled by each staff member, supporting performance review and shift planning.
SQL Query	<pre>SELECT st.staff_name, st.role, COUNT(DISTINCT o.order_id) AS orders_handled, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS revenue_handled, printf("%.2f", AVG(f.rating)) AS avg_feedback_rating FROM staff st JOIN orders o ON o.staff_id = st.staff_id JOIN order_item oi ON oi.order_id = o.order_id LEFT JOIN feedback f ON f.order_id = o.order_id WHERE o.status <> 'Cancelled' GROUP BY st.staff_id, st.staff_name, st.role ORDER BY SUM(oi.quantity * oi.unit_price) DESC;</pre>

Executed Output Screenshot

SQL Output Screenshot - F10

Staff Performance Report

staff_name	role	orders_handled	revenue_handled	avg_feedback_rating
Sadia Gill	Waiter	11	57057.00	4.10
Imran Butt	Waiter	8	32680.00	4.62
Rashid Khan	Delivery Rider	8	25519.00	4.00

Rows shown: 3 | Output generated from executed SQL on sample restaurant database.

Figure 11: Executed SQL output for F10 - Staff Performance Report (3 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.11 Feature F11: Payment Method Analysis

Breaks paid transactions into cash, card, and digital wallet amounts to support cashier reconciliation.

SQL Query

Field	Details
Feature ID	F11
Feature Name	Payment Method Analysis
Purpose	Breaks paid transactions into cash, card, and digital wallet amounts to support cashier reconciliation.
SQL Query	<pre>SELECT payment_method, COUNT(*) AS transactions, printf("%.2f", SUM(amount)) AS amount_collected, printf("%.2f", 100.0 * SUM(amount) / SUM(SUM(amount)) OVER ()) AS percent_of_total FROM payment WHERE payment_status = 'Paid' GROUP BY payment_method ORDER BY SUM(amount) DESC;</pre>

Executed Output Screenshot

SQL Output Screenshot - F11

Payment Method Analysis

payment_method	transactions	amount_collected	percent_of_total
Card	11	61087.00	55.44
Cash	7	22534.00	20.45
JazzCash	4	14475.00	13.14
Easypaisa	3	12090.00	10.97

Rows shown: 4 | Output generated from executed SQL on sample restaurant database.

Figure 12: Executed SQL output for F11 - Payment Method Analysis (4 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.12 Feature F12: Table Utilization Report

Shows which tables generate the most orders and revenue so managers can understand seating performance.

SQL Query

Field	Details
Feature ID	F12
Feature Name	Table Utilization Report
Purpose	Shows which tables generate the most orders and revenue so managers can understand seating performance.
SQL Query	<pre> SELECT dt.table_no, dt.capacity, dt.location, COUNT(DISTINCT o.order_id) AS served_orders, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS revenue_generated FROM dining_table dt JOIN orders o ON o.table_id = dt.table_id JOIN order_item oi ON oi.order_id = o.order_id WHERE o.status <> 'Cancelled' GROUP BY dt.table_id, dt.table_no, dt.capacity, dt.location ORDER BY COUNT(DISTINCT o.order_id) DESC, SUM(oi.quantity * oi.unit_price) DESC; </pre>

Executed Output Screenshot

SQL Output Screenshot - F12

Table Utilization Report

table_no	capacity	location	served_orders	revenue_generated
T07	6	Family Section	3	14490.00
T03	4	Main Hall	3	10940.00
T08	8	Private Room	2	18188.00
T06	6	Family Section	2	11020.00
T11	6	Main Hall	2	10279.00
T05	4	Patio	2	6680.00
T04	4	Main Hall	2	5270.00
T12	8	Private Room	1	5750.00
T10	4	Balcony	1	4000.00
T02	2	Window	1	3120.00

Rows shown: 10 | Output generated from executed SQL on sample restaurant database.

Figure 13: Executed SQL output for F12 - Table Utilization Report (10 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.13 Feature F13: Pending and Unpaid Order Monitoring

Identifies orders that are still open, preparing, or not fully paid so the cashier can follow up before closing.

SQL Query

Field	Details
Feature ID	F13
Feature Name	Pending and Unpaid Order Monitoring
Purpose	Identifies orders that are still open, preparing, or not fully paid so the cashier can follow up before closing.
SQL Query	<pre> SELECT o.order_id, c.customer_name, COALESCE(dt.table_no, 'Takeaway/Delivery') AS table_no, o.order_date, o.status AS order_status, printf("%.2f", SUM(oi.quantity * oi.unit_price)) AS bill_amount, COALESCE(p.payment_status, 'Not Paid') AS payment_status FROM orders o LEFT JOIN customer c ON c.customer_id = o.customer_id LEFT JOIN dining_table dt ON dt.table_id = o.table_id JOIN order_item oi ON oi.order_id = o.order_id LEFT JOIN payment p ON p.order_id = o.order_id WHERE o.status IN ('Open', 'Preparing') OR p.payment_status IS NULL OR p.payment_status <> 'Paid' GROUP BY o.order_id, c.customer_name, dt.table_no, o.order_date, o.status, p.payment_status ORDER BY o.order_date, o.order_id; </pre>

Executed Output Screenshot

SQL Output Screenshot - F13

Pending and Unpaid Order Monitoring

order_id	customer_name	table_no	order_date	order_status	bill_amount	payment_status
1011	Zain Qureshi	T01	2026-05-05	Cancelled	360.00	Not Paid
1018	Usman Tariq	T05	2026-05-10	Preparing	3070.00	Pending
1024	Ali Hassan	T04	2026-05-15	Open	2000.00	Pending

Rows shown: 3 | Output generated from executed SQL on sample restaurant database.

Figure 14: Executed SQL output for F13 - Pending and Unpaid Order Monitoring (3 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.14 Feature F14: Monthly Revenue Trend

Compares monthly sales and average bill size, giving management a high-level view of business growth.

SQL Query

Field	Details
Feature ID	F14
Feature Name	Monthly Revenue Trend
Purpose	Compares monthly sales and average bill size, giving management a high-level view of business growth.
SQL Query	<pre> WITH order_totals AS (SELECT o.order_id, o.order_date, SUM(oi.quantity * oi.unit_price) AS order_total FROM orders o JOIN order_item oi ON oi.order_id = o.order_id WHERE o.status <> 'Cancelled' GROUP BY o.order_id, o.order_date) SELECT strftime('%Y-%m', order_date) AS sales_month, COUNT(order_id) AS total_orders, printf('%2f', SUM(order_total)) AS monthly_revenue, printf('%2f', AVG(order_total)) AS average_bill FROM order_totals GROUP BY strftime('%Y-%m', order_date) ORDER BY sales_month; </pre>

Executed Output Screenshot

SQL Output Screenshot - F14

Monthly Revenue Trend

sales_month	total_orders	monthly_revenue	average_bill
2026-04	4	14889.00	3722.25
2026-05	19	81358.00	4282.00
2026-06	4	19009.00	4752.25

Rows shown: 3 | Output generated from executed SQL on sample restaurant database.

Figure 15: Executed SQL output for F14 - Monthly Revenue Trend (3 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

8.15 Feature F15: Customer Feedback Insights

Groups feedback by category to identify customer satisfaction patterns and improvement areas.

SQL Query

Field	Details
Feature ID	F15
Feature Name	Customer Feedback Insights
Purpose	Groups feedback by category to identify customer satisfaction patterns and improvement areas.
SQL Query	<pre>SELECT mc.category_name, COUNT(DISTINCT f.feedback_id) AS feedback_count, printf("%.2f", AVG(f.rating)) AS avg_rating, SUM(CASE WHEN f.rating <= 3 THEN 1 ELSE 0 END) AS improvement_alerts FROM feedback f JOIN orders o ON o.order_id = f.order_id JOIN order_item oi ON oi.order_id = o.order_id JOIN menu_item mi ON mi.item_id = oi.item_id JOIN menu_category mc ON mc.category_id = mi.category_id GROUP BY mc.category_name ORDER BY AVG(f.rating) ASC, feedback_count DESC;</pre>

Executed Output Screenshot

SQL Output Screenshot - F15

Customer Feedback Insights

category_name	feedback_count	avg_rating	improvement_alerts
Deals	4	4.00	0
Rice	8	4.13	2
BBQ	8	4.25	2
Beverages	15	4.33	1
Desserts	9	4.33	2
Main Course	9	4.33	1
Starters	3	4.67	0

Rows shown: 7 | Output generated from executed SQL on sample restaurant database.

Figure 16: Executed SQL output for F15 - Customer Feedback Insights (7 rows returned)

Interpretation: The output converts raw database records into useful information for restaurant staff and management decision-making.

9. Final Summary of the Restaurant Management System

The Restaurant Management System database project successfully models the important operations of a modern restaurant. It organizes information about customers, staff, tables, reservations, menus, orders, payments, suppliers, inventory, purchase orders, and feedback in a structured relational database. The design reduces duplicate data and supports accurate reporting through primary key and foreign key relationships.

The Phase 3 queries prove that the database can do more than store records. It can generate real operational reports such as active menu lists, table reservation schedules, live bill calculations, daily sales reports, best-selling items, category revenue, customer loyalty rankings, low-stock alerts, supplier summaries, staff performance reports, payment analysis, table utilization, pending payment tracking, monthly revenue trends, and feedback insights.

From a business perspective, this system helps managers make faster and better decisions. Sales reports show which items and categories produce the highest revenue. Inventory alerts protect the restaurant from running out of ingredients. Payment reports help cashiers reconcile collections. Staff reports support fair performance review. Feedback reports help improve customer satisfaction and service quality.

9.1 Future Enhancement Scope

- Add a login and role-based access control system for managers, cashiers, waiters, and kitchen staff.
- Connect the database to a web or desktop front end for real-time order entry and payment processing.
- Add automatic stock deduction when orders are completed.
- Create invoice printing and digital receipt features.
- Add dashboard charts for revenue, inventory, staff performance, and feedback trends.
- Add online reservation and delivery integration for customers.

Overall, the project demonstrates how a well-designed database can support restaurant efficiency, accuracy, customer service, and management control.

GITHUB LINK:

(https://github.com/70154063-ai/Restaurant_Management)

THANK YOU!